

Service Accounts in Kubernetes

Since you're learning **KCNA as a beginner**, let's understand Service Accounts in a very easy way.

1. What is a Service Account?

A ServiceAccount is an identity used by applications, Pods, or other workloads to interact with the Kubernetes API.

The easiest way to remember:

 **User account** → for humans

 **ServiceAccount** → for applications/PODs

2. Why Do We Need Service Accounts?

Imagine a Pod needs to talk to the Kubernetes API.

For example, an application inside a Pod wants to:

Get information about Pods

Read ConfigMaps

Check Services

Kubernetes needs to know:

"Who is making this request?"

The Pod can use a **ServiceAccount** as its identity.

Pod 

|

| uses



ServiceAccount 

|



Kubernetes API Server

3. Human User vs Service Account ★

This is very important.

Human

Abeera



User Account



kubectl



API Server

Application

Application



Pod



ServiceAccount



API Server

Easy memory:



Human = User identity



Application = ServiceAccount identity

4. Real-Life Example 🏢

Imagine a company.

Human employee

A person has an:

Employee ID

This identifies the person.

Application

A machine/application has:

Service ID

This identifies the application.

Similarly:

Human → User

Application → ServiceAccount

5. How Does a Pod Use a ServiceAccount?

A Pod can specify:

spec:

serviceAccountName: my-service-account

Example:

apiVersion: v1

kind: Pod

metadata:

name: my-pod

spec:

serviceAccountName: my-service-account

containers:

- name: nginx

image: nginx

This means:

"Run this Pod using the my-service-account identity."

6. Creating a ServiceAccount

You can create one using:

```
kubectl create serviceaccount my-service-account
```

Check it:

```
kubectl get serviceaccounts
```

or:

```
kubectl get sa
```

sa = ServiceAccount.

7. Describe a ServiceAccount

```
kubectl describe serviceaccount my-service-account
```

8. ServiceAccount and RBAC ★ ★ ★

This is VERY important because it connects to the topics you've just learned.

A ServiceAccount by itself doesn't automatically have broad permissions.

You can give it permissions using **RBAC**.

For example:

ServiceAccount

|



RoleBinding



Role



Permissions

So an application can be given **only the permissions it needs**.

9. Example

Suppose an application needs to read Pods.

We can create:

ServiceAccount



RoleBinding



Role



Now the application can access Pods according to those permissions.

It doesn't automatically get permission to:

✗ Delete Nodes

✗ Modify cluster settings

✗ Delete other resources

This follows the security principle of **least privilege**.

10. Default ServiceAccount

Every Kubernetes namespace normally has a ServiceAccount called:

default

Check it:

kubectl get serviceaccounts

You may see:

NAME SECRETS

default ...

If you create a Pod and don't specify:

serviceAccountName:

the Pod normally uses the namespace's:

default

ServiceAccount.

Easy memory:

No ServiceAccount specified → default ServiceAccount is normally used.

11. ServiceAccount Flow

Remember this diagram:

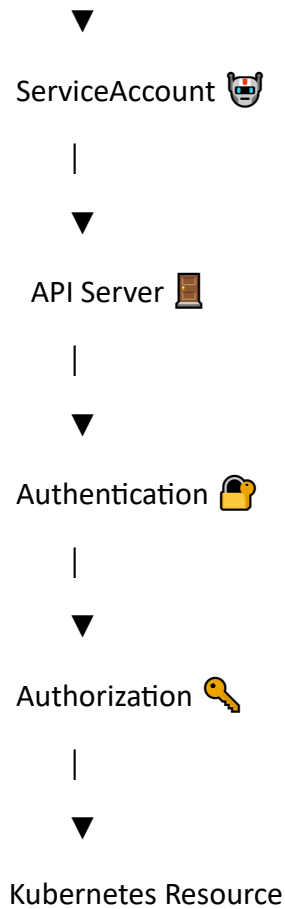
Application

|



Pod 

|



12. ServiceAccount + Authentication

Remember your previous topic:

Authentication

Who are you?

A ServiceAccount provides an identity for a workload.

For example:

API Server:

"Who is requesting this?"

Answer:

"my-service-account"

Then authorization checks:

"What is this ServiceAccount allowed to do?"

RBAC can answer that.

13. ServiceAccount + RBAC ★

This is one of the most important relationships:

ServiceAccount

|

|



RoleBinding

|



Role

|



Permissions

Example:

my-app

|



ServiceAccount

|



RoleBinding

|



pod-reader

|

├─ get Pods

├─ list Pods

└─ watch Pods

14. ServiceAccount vs User Account

User Account

Represents humans

Used by people

Example: developer

Often used with kubectl

ServiceAccount

Represents applications/workloads

Used by Pods/applications

Example: backend application

Used by workloads to access APIs

Easy Trick

 User Account



Human

 ServiceAccount



Application

15. Important Security Point

Don't give a ServiceAccount unnecessary permissions.

Bad approach:

Application



Administrator permissions ❌

Better:

Application



Only required permissions ✅

For example, if an application only needs to **read Pods**, give it only:

get

list

watch

for Pods.

This is called:

Principle of Least Privilege

16. Useful Commands ★

Create

```
kubectl create serviceaccount my-sa
```

List

```
kubectl get serviceaccounts
```

or:

```
kubectl get sa
```

Describe

```
kubectl describe serviceaccount my-sa
```

Check ServiceAccount in a Pod

```
kubectl get pod my-pod -o yaml
```

Look for:

serviceAccountName: my-sa

KCNA Revision Notes

ServiceAccount

- Provides an **identity for applications/workloads**.
- Usually used by **Pods** to access the Kubernetes API.
- Different from a human user identity.
- A Pod can specify a ServiceAccount using:

serviceAccountName: my-sa

- Every namespace normally has a **default ServiceAccount**.
 - ServiceAccounts can be given permissions using **RBAC**.
 - Use **least privilege**: give only the permissions the application needs.
-

Super Easy Memory

USER



Human



kubectl



API Server

SERVICE ACCOUNT



Application / Pod



API Server

And:

ServiceAccount



RBAC



Permissions

One-line exam definition:

A ServiceAccount is a Kubernetes identity used by Pods and applications to authenticate to the Kubernetes API and access resources according to their assigned permissions.